

# 计算机图形学

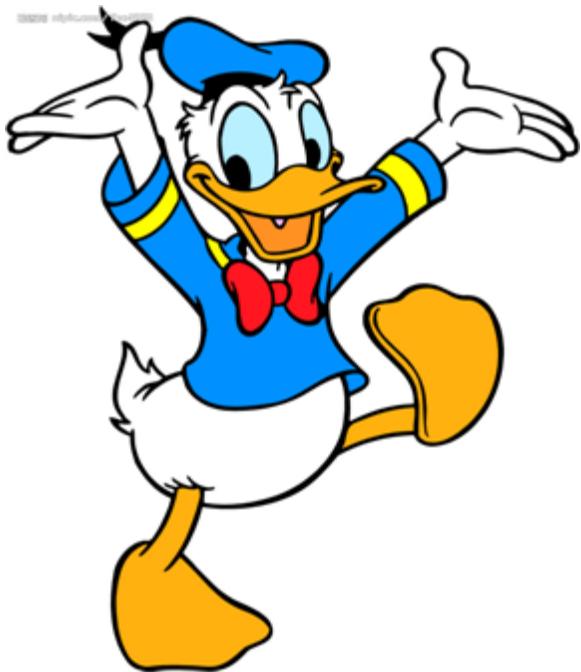
## 第二章：光栅图形学算法

# 光栅图形学算法的研究内容

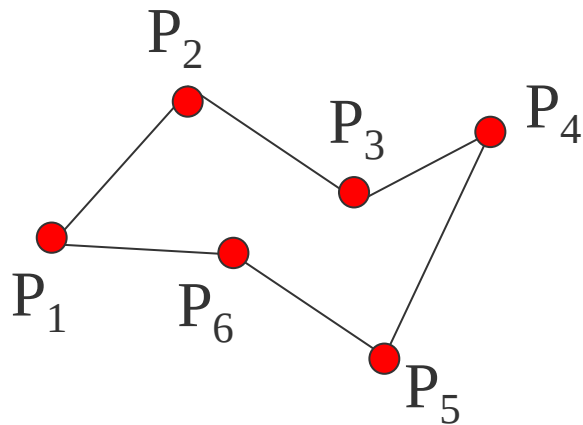
- 直线段的扫描转换算法
- 多边形的扫描转换与区域填充算法
- 裁剪算法
- 反走样算法
- 消隐算法

# 一、多边形的扫描转换

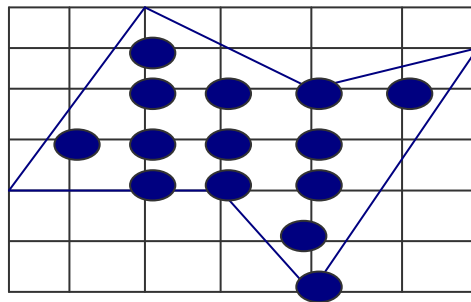
多边形的扫描转换和区域填充这个问题是怎么样在离散的像素集上表示一个连续的**二维图形**



多边形有两种重要的表示方法：**顶点**表示和**点阵**表示



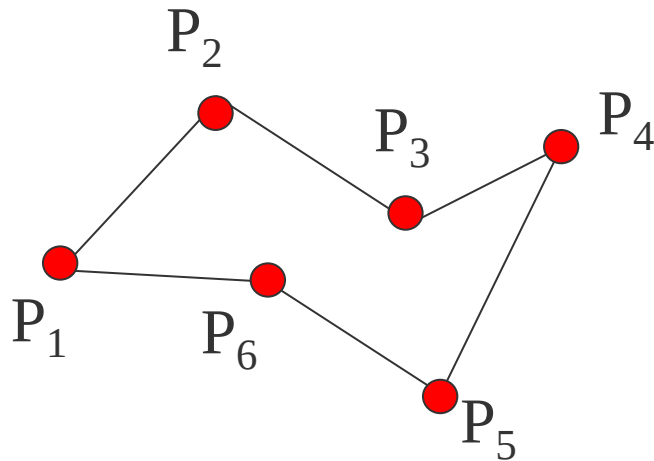
顶点表示



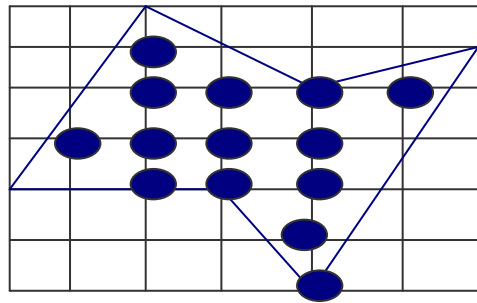
点阵表示

**顶点**表示是用多边形的顶点序列来表示多边形。这种表示直观、几何意义强、占内存少，易于进行几何变换

但由于它没有明确指出哪些像素在多边形内，故不能直接用于面着色

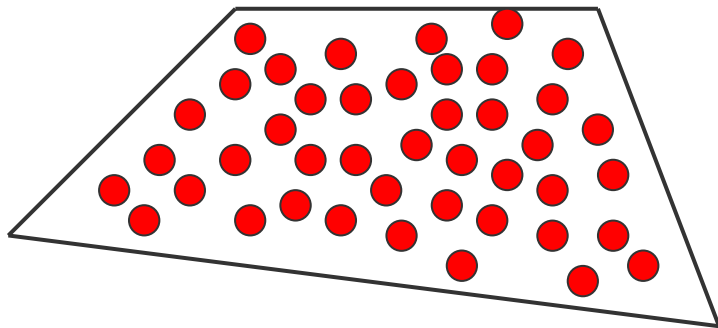


**点阵**表示是用位于多边形内的象  
素集合来刻画多边形。这种表示  
丢失了许多几何信息（如**边界**、  
**顶点**等），但它却是光栅显示系  
统显示时所需的表示形式。

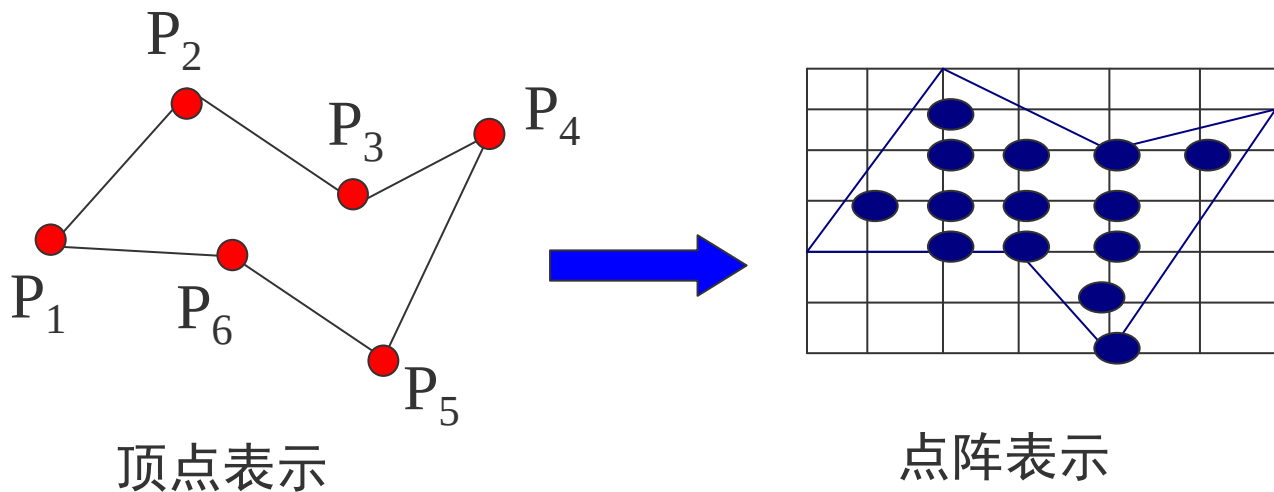


这涉及到两个问题：**第一个问题**是如果知道边界，能否求出**哪些像素**在多边形内？

第二个问题是知道多边形内部的像素，反过来如何求多边形的边界？



光栅图形的一个基本问题是把多边形的**顶点**表示转换为**点阵**表示。这种转换称为**多边形的扫描转换**

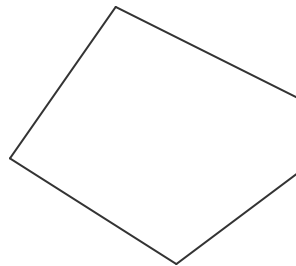




多边形分为凸多边形、凹多边形、含内环的多边形等：

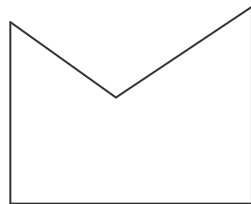
(1) 凸多边形

任意两顶点间的连线均在多边形内



(2) 凹多边形

任意两顶点间的连线有不在在多边形内

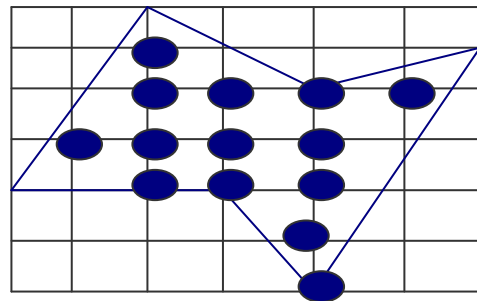
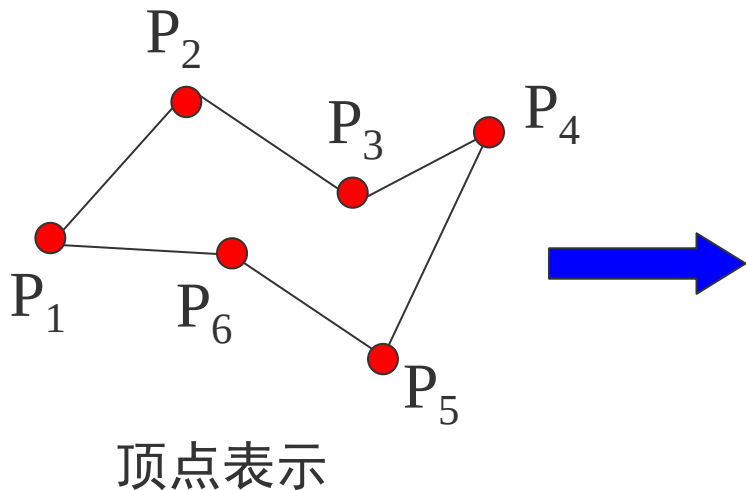


(3) 含内环的多边形

多边形内包含多边形



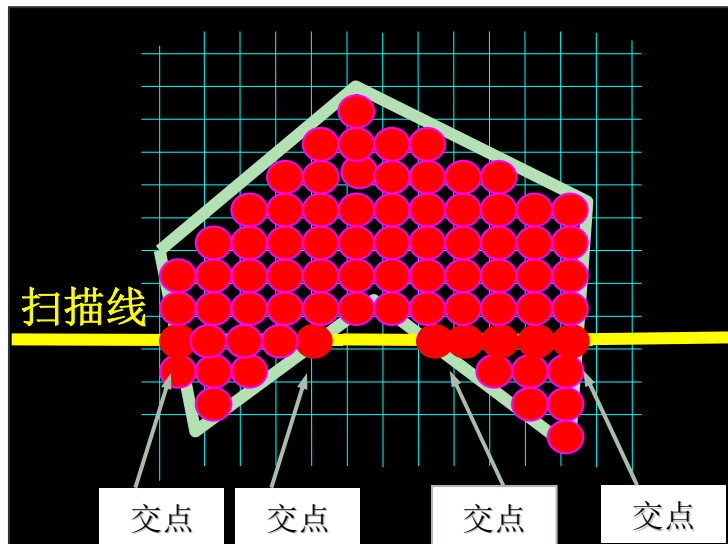
现在的问题是，知道多边形的边界，如何找到多边形内部的点，即把多边形内部涂上颜色



# 1、X-扫描线算法

**X-扫描线**算法填充多边形的基本思想是按扫描线顺序，计算扫描线与多边形的相交区间，再用要求的颜色显示这些区间的像素，即完成填充工作

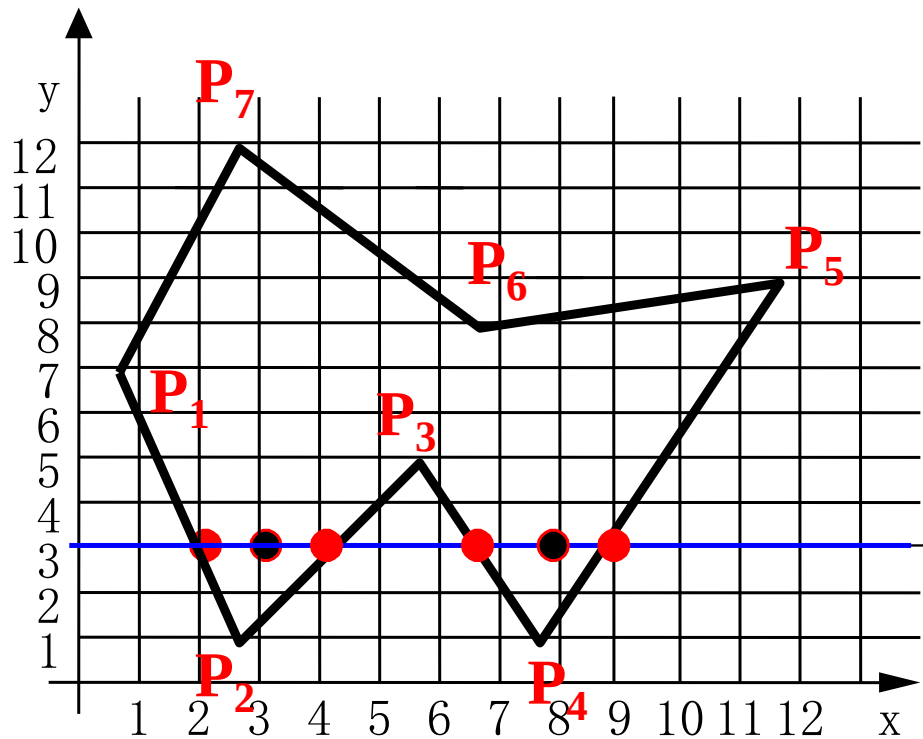
区间的端点可以通过计算扫描线与多边形边界线的交点获得



如扫描线 $y=3$ 与多边形的边界相交于4点：

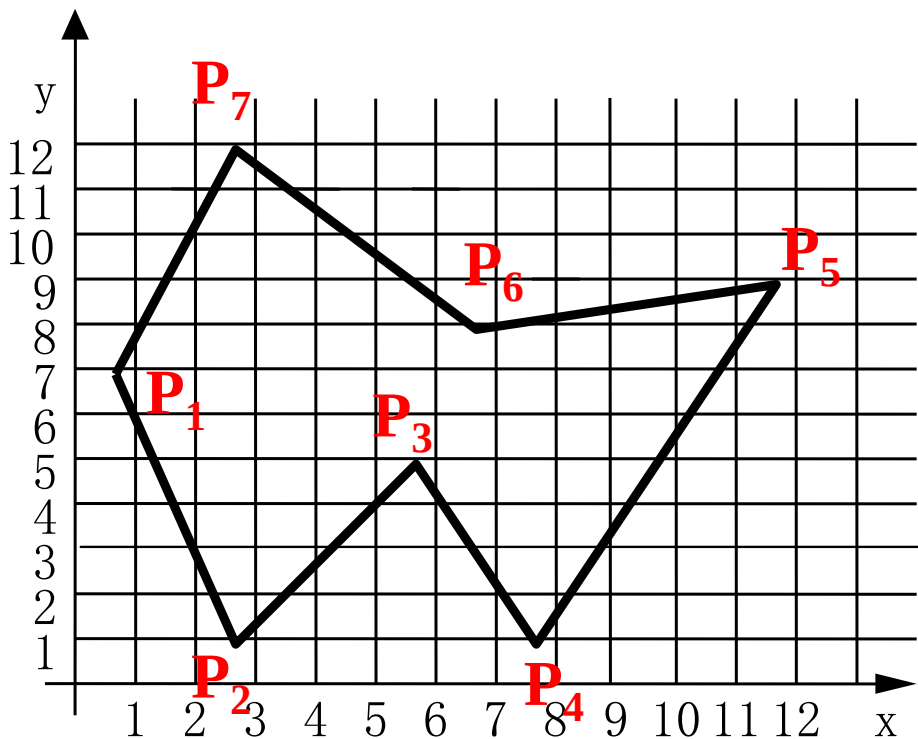
$(2, 3)$ 、 $(4, 3)$ 、 $(7, 3)$   
、 $(9, 3)$ 。

这四点定义了扫描线从 $x=2$ 到  
 $x=4$ ，从 $x=7$ 到 $x=9$ 两个落在多  
边形内的区间，该区间内的  
像素应取填充色



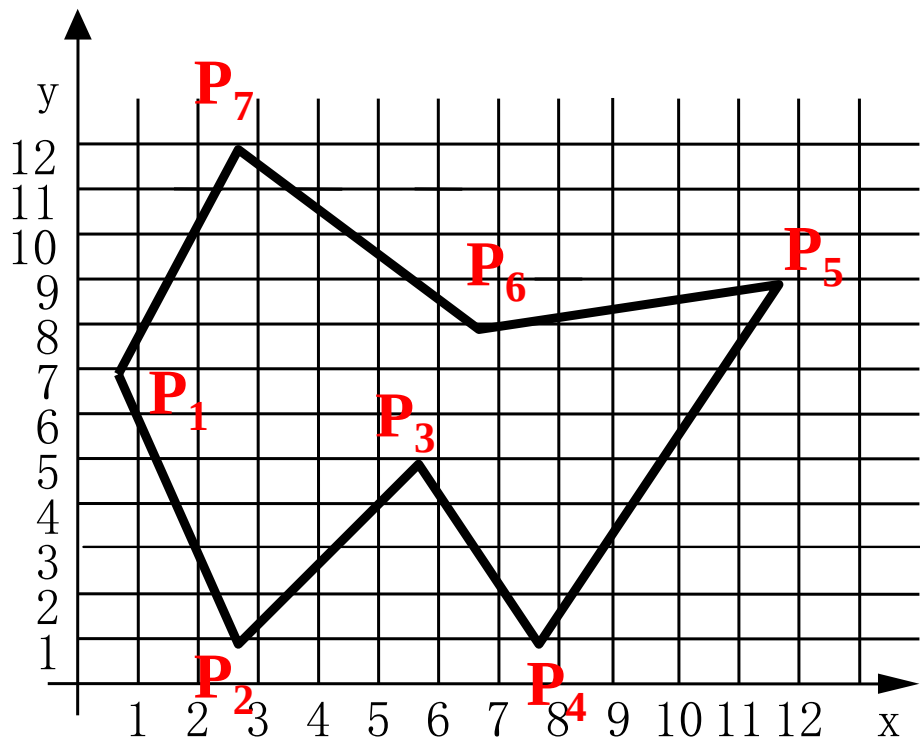
算法的核心是按X递增顺序排列交点的X坐标序列。由此，可得到X-扫描线算法步骤如下：

(1) 确定多边形所占有的最大扫描线数，得到多边形顶点的最小和最大y值 ( $y_{\min}$  和  $y_{\max}$ )



算法的核心是按X递增顺序排列交点的X坐标序列。由此，可得到X-扫描线算法步骤如下：

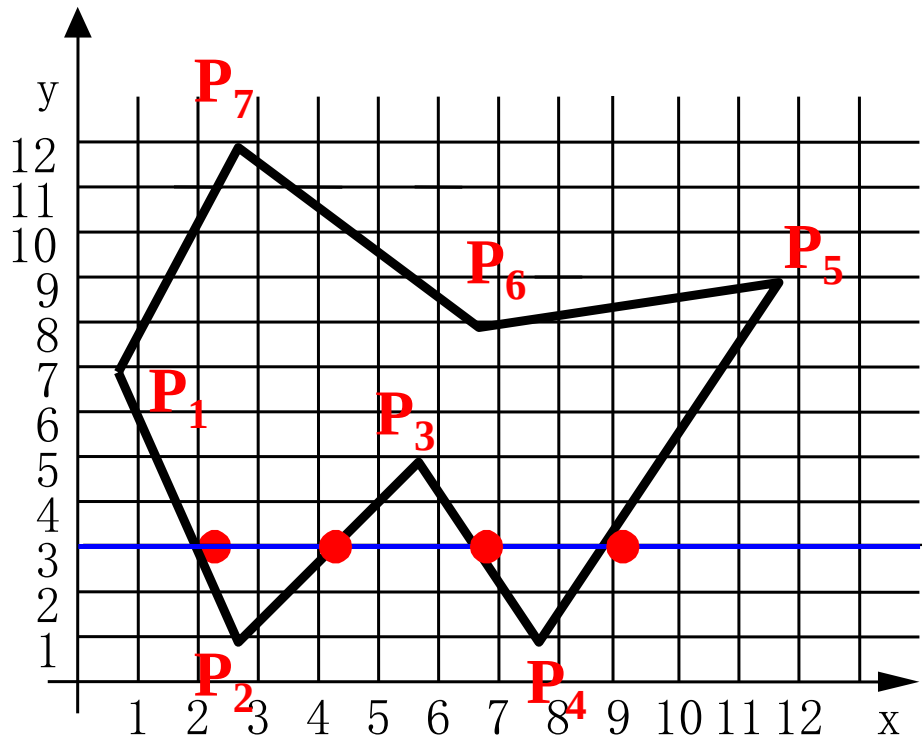
(2) 从 $y = y_{\min}$ 到 $y = y_{\max}$ ，  
每次用一条扫描线进行填充

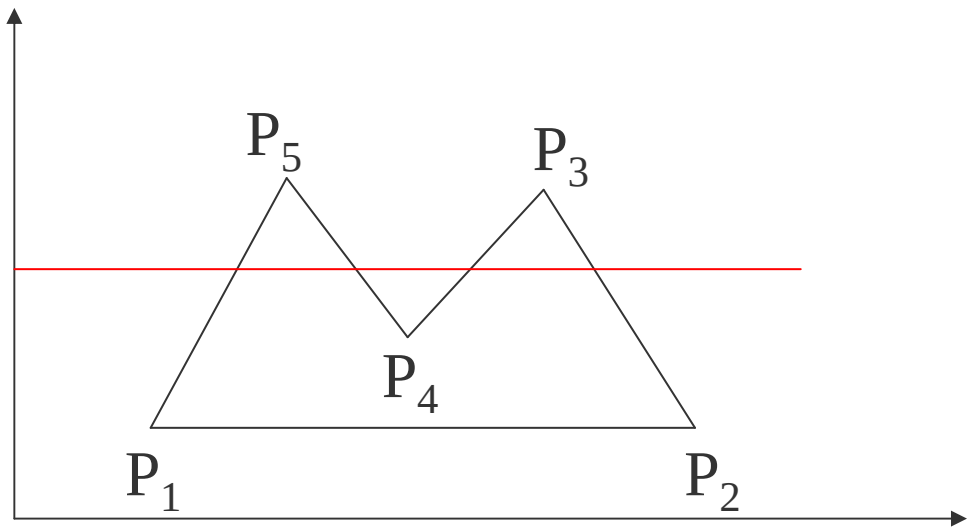


算法的核心是按X递增顺序排列交点的X坐标序列。由此，可得到X-扫描线算法步骤如下：

(3) 对一条扫描线填充的过程可分为四个步骤：

- a、求交：计算扫描线与多边形各边的交点
- b、排序：把所有交点按递增顺序进行排序





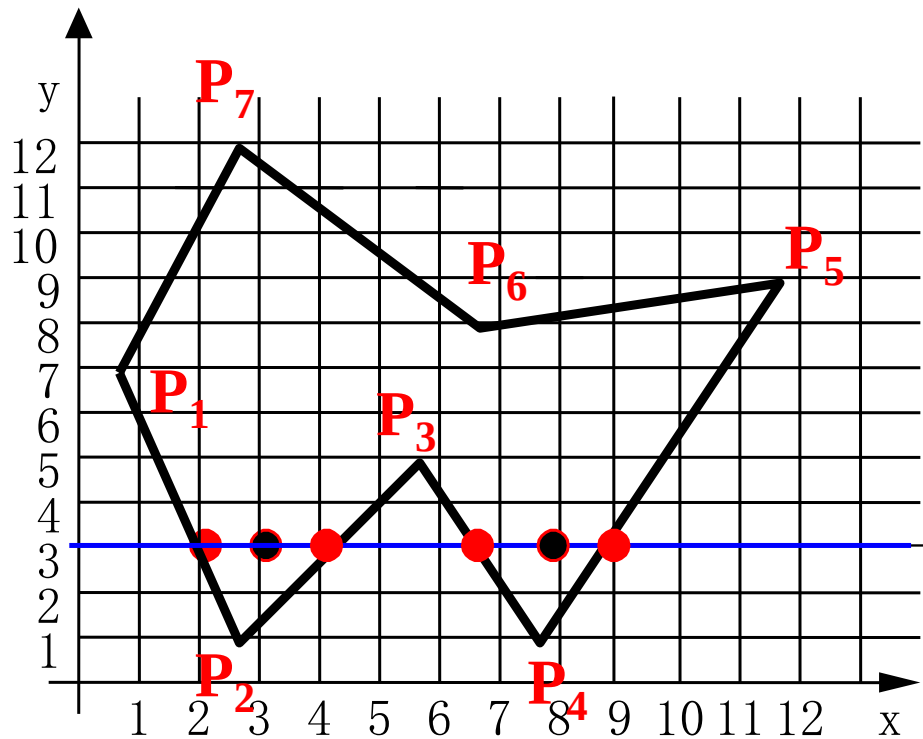


算法的核心是按X递增顺序排列交点的X坐标序列。由此，可得到X-扫描线算法步骤如下：

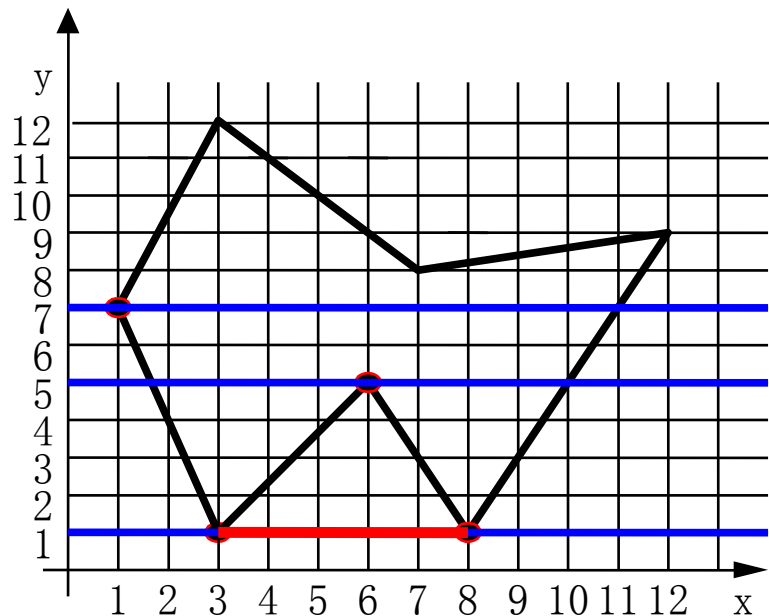
(3) 对一条扫描线填充的过程可分为四个步骤：

c、交点配对：第一个与第二个，第三个与第四个

d、区间填色：把这些相交区间内的像素置成不同于背景色的填充色



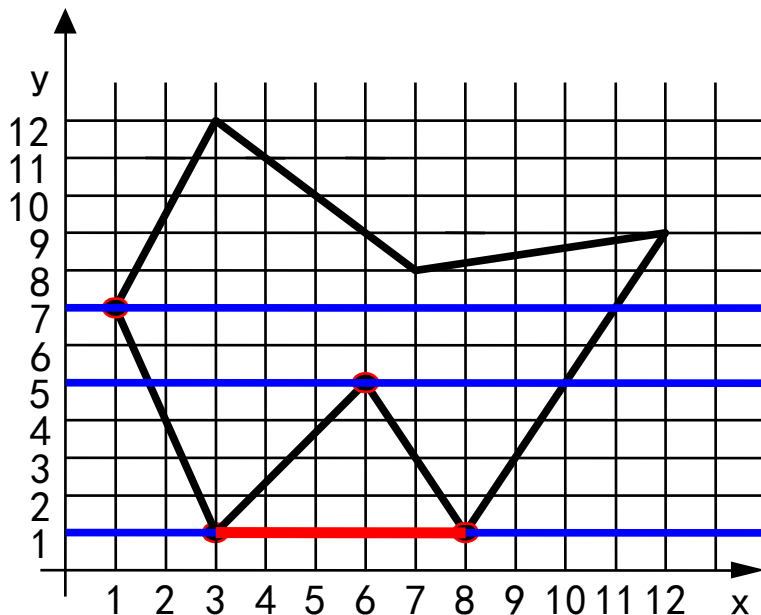
当扫描线与多边形顶点相交时，交点的取舍问题（交点的个数应保证为偶数个）



## 解决方案：

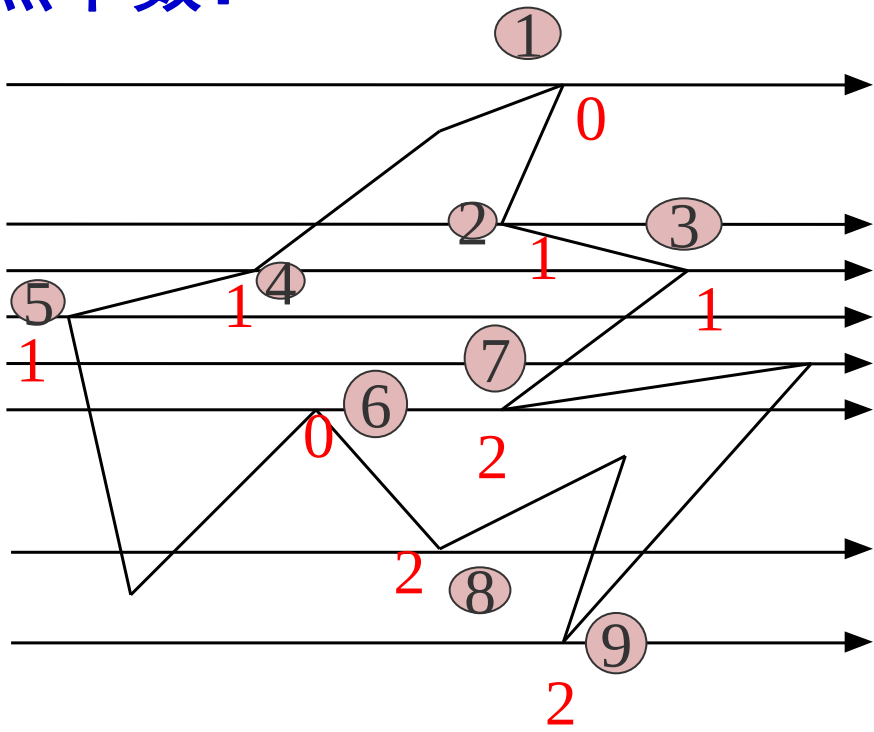
(1) 若共享顶点的两条边分别落在扫描线的两边，交点只算一个

(2) 若共享顶点的两条边在扫描线的同一边，这时交点作为  
**零**个或**两**个



检查共享顶点的两条边的另外两个端点的 $y$ 值，按这两个 $y$ 值中大于交点 $y$ 值的个数来决定交点数

举例计算交点个数：



为了计算每条扫描线与多边形各边的交点，最简单的方法是把多边形的所有边放在一个表中。在处理每条扫描线时，**按顺序从表中取出所有的边，分别与扫描线求交**

这个算法效率低，为什么？

关键问题是**求交**！而求交是很可怕的，求交的计算量是非常大的